```
import numpy as np


def parallel_cellular_segmentation(image_matrix, num_seeds=3, max_iterations=200):
    """Segments a 2D pixel matrix into a 2D label matrix using a parallel cellular algorithm."""
    # 1. Normalize input pixel matrix to [0.0, 1.0]
    img = np.array(image_matrix, dtype=np.float32)
    img = (img - img.min()) / (img.max() - img.min() + 1e-8)
    H, W = img.shape

    # 2. Initialize label and strength matrices
    labels = np.zeros((H, W), dtype=np.int32)
    strength = np.zeros((H, W), dtype=np.float32)

    # 3. Place random starting seeds across the matrix
    np.random.seed(42)  # Fixed seed for reproducible results
    rand_rows = np.random.randint(0, H, size=num_seeds)
    rand_cols = np.random.randint(0, W, size=num_seeds)

    for i in range(num_seeds):
        labels[rand_rows[i], rand_cols[i]] = i + 1
        strength[rand_rows[i], rand_cols[i]] = 1.0

    # Define 8-neighbor offsets (Moore neighborhood)
    neighbors = [
        (-1, -1), (-1, 0), (-1, 1),
        (0, -1),           (0, 1),
        (1, -1),  (1, 0),  (1, 1)
    ]

    # 4. Main Parallel Evolution Loop
    for iteration in range(max_iterations):
        next_labels = labels.copy()
        next_strength = strength.copy()

        # Pad matrices to safely check boundary pixels in parallel
        pad_img = np.pad(img, 1, mode='edge')
        pad_labels = np.pad(labels, 1, mode='constant', constant_values=0)
        pad_strength = np.pad(strength, 1, mode='constant', constant_values=0.0)

        # Evaluate all 8 neighbors simultaneously across the whole matrix
        for dr, dc in neighbors:
            n_img = pad_img[1 + dr : H + 1 + dr, 1 + dc : W + 1 + dc]
            n_labels = pad_labels[1 + dr : H + 1 + dr, 1 + dc : W + 1 + dc]
            n_strength = pad_strength[1 + dr : H + 1 + dr, 1 + dc : W + 1 + dc]

            # Calculate brightness similarity and attack force
            similarity = 1.0 - np.abs(img - n_img)
            attack_force = n_strength * similarity

            # Conquer pixels where neighbor attack beats current strength
            conquered = attack_force > next_strength
            next_strength[conquered] = attack_force[conquered]
            next_labels[conquered] = n_labels[conquered]

        # Stopping condition: exit if no cell changed its label
        if np.array_equal(labels, next_labels):
            print(f"Converged at iteration: {iteration + 1}")
            labels = next_labels
            break

        labels = next_labels
        strength = next_strength

    return labels, strength


# ========================================
# EXAMPLE USAGE (PURE MATRIX INPUT & OUTPUT)
# ========================================
if __name__ == "__main__":
    # Input: A 8x8 pixel intensity matrix (0.1 = dark background, 0.5 = tissue, 0.9 = tumor)
    pixel_matrix = [
        [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1],
        [0.1, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.1],
        [0.1, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.1],
        [0.1, 0.5, 0.5, 0.9, 0.9, 0.5, 0.5, 0.1],
        [0.1, 0.5, 0.5, 0.9, 0.9, 0.5, 0.5, 0.1],
        [0.1, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.1],
        [0.1, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.1],
        [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]
```

```
    ]

    # Run the algorithm
    label_matrix, confidence_matrix = parallel_cellular_segmentation(
        pixel_matrix, num_seeds=3
    )

    # Print pure matrix outputs
    print("\n--- FINAL SEGMENTED LABEL MATRIX ---")
    print(label_matrix)

    print("\n--- PIXEL COUNT PER SEGMENT LABEL ---")
    unique_labels, counts = np.unique(label_matrix, return_counts=True)
    for lbl, count in zip(unique_labels, counts):
        print(f"Region Label {lbl}: {count} pixels")
```

```
Converged at iteration: 13

--- FINAL SEGMENTED LABEL MATRIX ---
[[3 3 3 3 3 3 3]
 [3 2 2 2 2 2 3]
 [3 2 2 2 2 2 3]
 [3 2 2 2 2 1 3]
 [3 2 2 2 2 1 3]
 [3 2 2 2 1 1 3]
 [3 2 2 2 1 1 3]
 [3 3 3 3 3 3 3]]

--- PIXEL COUNT PER SEGMENT LABEL ---
Region Label 1: 9 pixels
Region Label 2: 27 pixels
Region Label 3: 28 pixels
```